

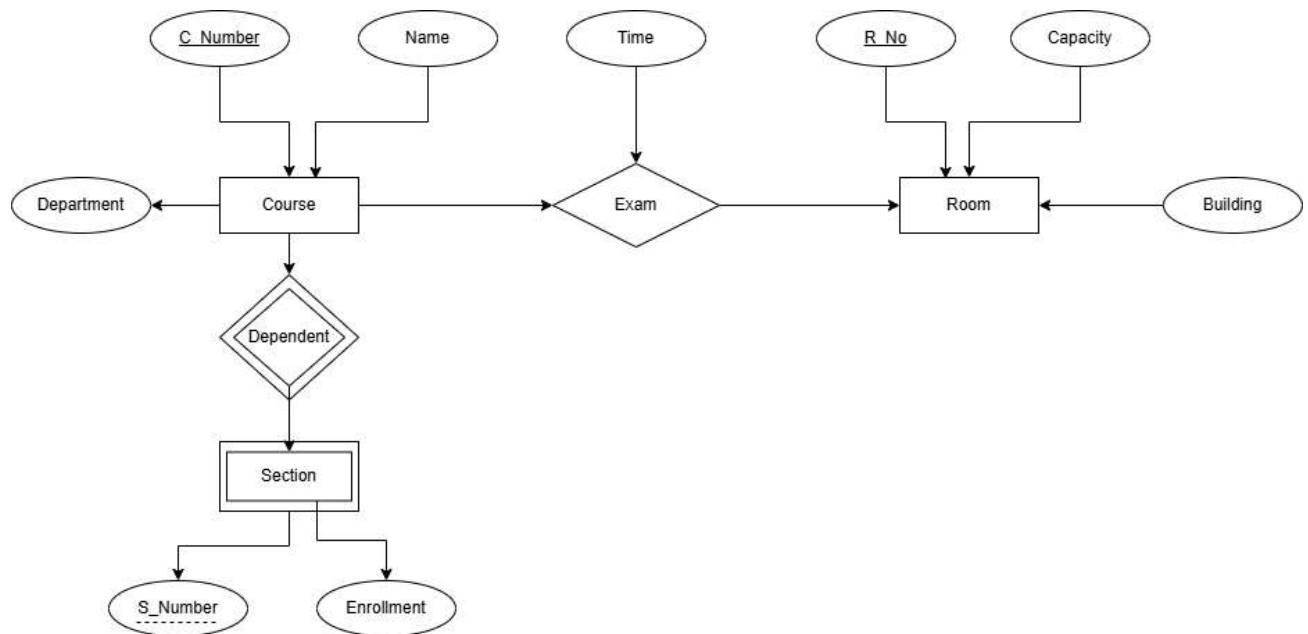
EXPERIMENT 2

Aim :- Creating ER Diagram using Case Tools

Case Study :-

Consider a university database for the scheduling of classrooms for final exams. This database could be modelled as the single entity set exam, with attributes course-name, section- number, room-number, and time. Alternatively, one or more additional entity sets could be defined, along with relationship sets to replace some of the attributes of the exam entity set, as

- Course with attributes name, department, and c-number
- Section with attributes s-number and enrolment, and dependent as a weak entity set on course
- Room with attributes r-number, capacity, and building



Case Study :-

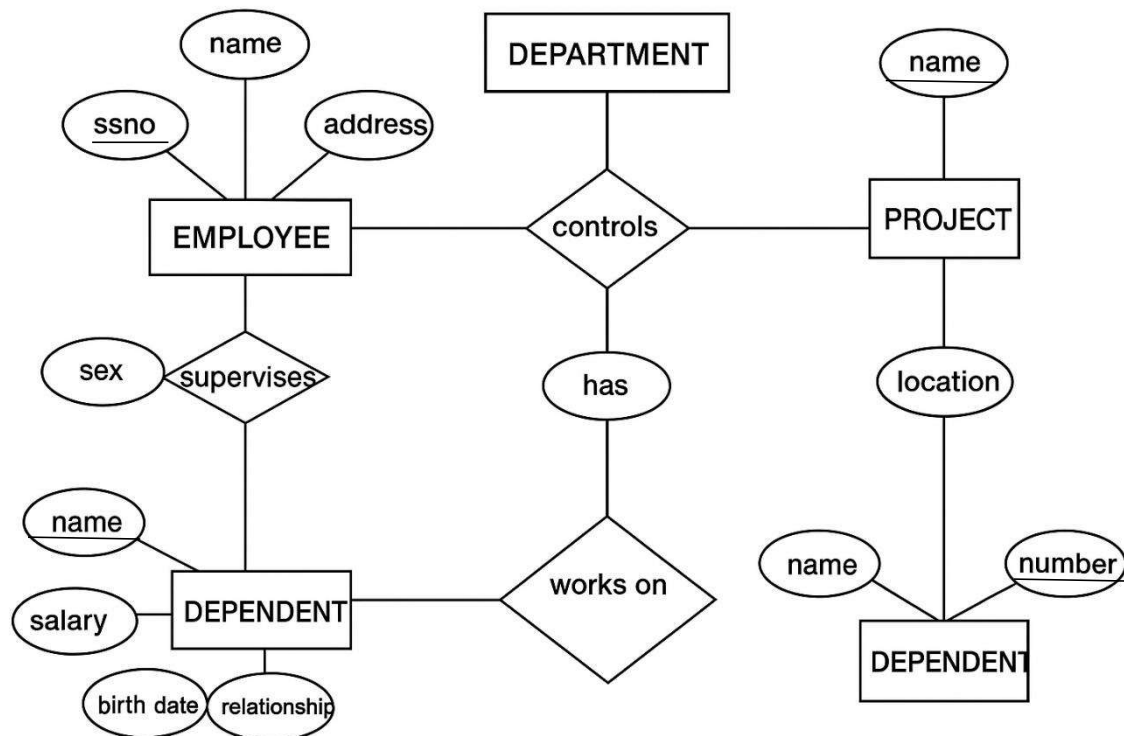
Construct an ER Diagram for Company having following details: • Company organized into DEPARTMENT.

Each department has unique name and a particular employee who manages the department. Start date for the manager is recorded. Department may have several locations.

- A department controls a number of PROJECT. Projects have a unique name, number and a single location.

- Company's EMPLOYEE name, ssno, address, salary, sex and birth date are recorded. An employee is assigned to one department, but may work for several projects (not necessarily controlled by her dept). Number of hours/week an employee works on each project is recorded; the immediate supervisor for the employee.

- Employee's DEPENDENT are tracked for health insurance purposes (dependent name, birthdate, relationship to employee).



EXPERIMENT 3

Aim :- Execution of DDL Commands and Constraints

1. **Complete the GRADUATE CANDIDATE table instance chart. Credits is a foreign-key column referencing the requirements table.**

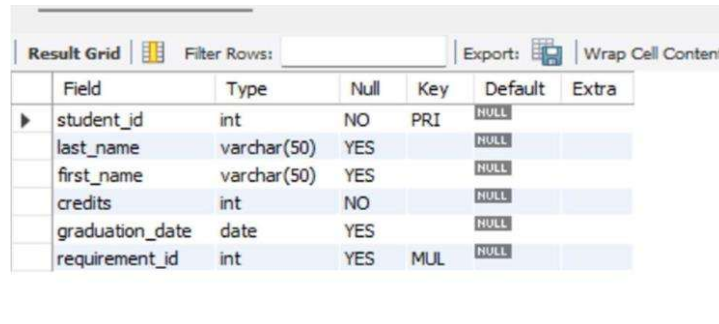
Column Name	student id	last name	first name	credits	Graduation date
Key Type	Primary Key			Foreign Key	
Nulls/Unique	Not Null / Unique	Not Null	Not Null	Not Null	Nullable
FK Column				References REQUIREMENTS(credits)	
Datatype	NUMBER	VARCHAR2	VARCHAR2	NUMBER	DATE
Length	6	15	15	3	

2. **Write the syntax to create the grad_candidates table.**

```
CREATE TABLE grad_candidates (  
  student_id INT(6) PRIMARY Key,  
  last_name VARCHAR(50),  
  first_name VARCHAR(50), credits  
  INT(3) NOT NULL,  
  graduation_date DATE,  
  requirement_id INT,  
  CONSTRAINT _requirement  
    FOREIGN KEY (requirement_id)  
    REFERENCES requirements(requirement_id)  
);
```

3. **Confirm creation of the table.**

DESC grad_candidates;



The screenshot shows a database interface with a 'Result Grid' tab. It displays the structure of the 'grad_candidates' table. The table has 7 columns: Field, Type, Null, Key, Default, and Extra. The rows are: student_id (int, NO, PRI, NULL), last_name (varchar(50), YES, NULL), first_name (varchar(50), YES, NULL), credits (int, NO, NULL), graduation_date (date, YES, NULL), and requirement_id (int, YES, MUL, NULL).

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	
last_name	varchar(50)	YES		NULL	
first_name	varchar(50)	YES		NULL	
credits	int	NO		NULL	
graduation_date	date	YES		NULL	
requirement_id	int	YES	MUL	NULL	

4. **Create a new table using grad_candidates with the following syntax:**
CREATE TABLE new_grad_candidates AS (SELECT * FROM grad_candidates);

```
CREATE TABLE new_grad_candidates AS  
SELECT * FROM grad_candidates;
```



The screenshot shows a query log with two entries. The first entry is a DESCRIBE statement for grad_candidates, which returned 6 rows. The second entry is the CREATE TABLE statement for new_grad_candidates, which executed successfully with 0 rows affected and no warnings.

Query ID	Time	Query	Rows	Records	Duplicates	Warnings	Time
8	11:53:22	DESC grad_candidates	6 row(s) returned				0.000 sec / 0.000 sec
9	11:54:10	CREATE TABLE new_grad_candidates AS SELECT * FROM grad_candi...	0 row(s) affected	0	0	0	0.031 sec

5. **Create a new table using a subquery. Name the new table your first name e.g., gaurav_table. Using a subquery, copy grad_candidates into gaurav_table.**

```
CREATE TABLE wasim_table AS
```

```
SELECT * FROM grad_candidates;
```



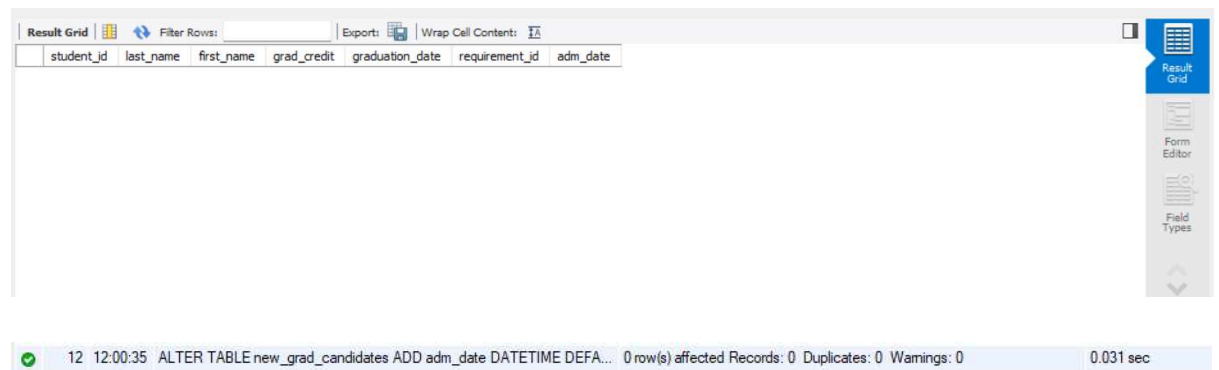
The screenshot shows a query log with two entries. The first entry is the CREATE TABLE statement for new_grad_candidates, which executed successfully with 0 rows affected. The second entry is the CREATE TABLE statement for wasim_table, which also executed successfully with 0 rows affected.

Query ID	Time	Query	Rows	Records	Duplicates	Warnings	Time
21	11:26:15	CREATE TABLE new_grad_candidates AS SELECT * FROM grad_candidates	0 row(s) affected	0	0	0	0.047 sec
22	11:26:30	CREATE TABLE wasim_table AS SELECT * FROM grad_candidates	0 row(s) affected	0	0	0	0.046 sec

6. In your new_grad_candidates table, enter a new column called "adm_date." The datatype for the new column should be VARCHAR2. Set the DEFAULT for this column as SYSDATE.

```
ALTER TABLE new_grad_candidates
```

```
ADD adm_date DATETIME DEFAULT NOW();
```



7. In your new_grad_candidates table, increase the length of last_name column by 10 and remove the credits column.

```
ALTER TABLE new_grad_candidates
```

```
MODIFY last_name VARCHAR(30);
```

```
ALTER TABLE new_grad_candidates
```

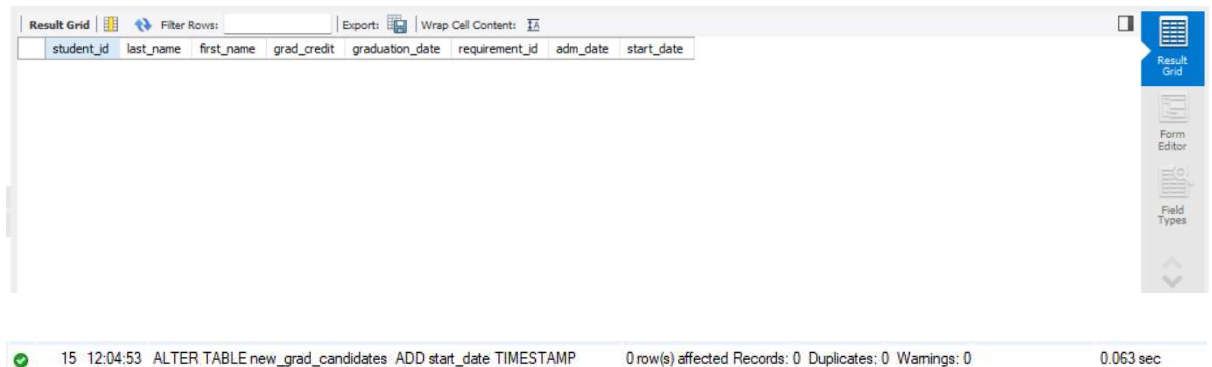
```
DROP COLUMN credits;
```



8. **Create a new column in the new_grad_candidates table called start_date. Use the TIMESTAMP WITH LOCAL TIME ZONE as the datatype.**

```
ALTER TABLE new_grad_candidates
```

```
ADD start_date TIMESTAMP;
```



9. **Write syntax to change the name of credit column by grad_credit.**

```
ALTER TABLE grad_candidates RENAME
```

```
COLUMN credits TO grad_credit;
```



10. **Insert 5 tuples in wasim_table.**

```
INSERT INTO wasim_table (student_id, last_name, first_name, credits, graduation_date)
```

```
VALUES (101, 'Ansari', 'Wasim', 90, STR_TO_DATE('30-05-2025', '%d-%m-%Y'));
```

```
INSERT INTO wasim_table (student_id, last_name, first_name, credits, graduation_date)
```

```
VALUES (102, 'Tripathi', 'Kushagra', 92, STR_TO_DATE('12-06-2025', '%d-%m-%Y'));
```

```
INSERT INTO wasim_table (student_id, last_name, first_name, credits, graduation_date)
```

```
VALUES (103, 'Kumar', 'Rahul', 88, STR_TO_DATE('24-05-2025', '%d-%m-%Y'));
```

```
INSERT INTO wasim_table (student_id, last_name, first_name, credits, graduation_date)
```

VALUES (105, 'Kumar', 'Yash', 89, STR_TO_DATE('08-07-2025', '%d-%m-%Y'));

12. What the distinction is between TRUNCATE and DROP for tables?

TRUNCATE	DROP
Removes all rows quickly	Delete whole table
Structure exists	Structure does not exists
removes the table definition.	keeps the table definition and columns.
Cannot be rolled back (once dropped, the table is gone permanently).	Can be rolled back <i>if</i> used inside a transaction.

13. List the changes that can and cannot be made to a column.

Changes can be done	Changes cannot be done
Increase column length	Decrease column size
Add new columns	Change datatype
Rename columns	Add NOT NULL to column null values
Modify default values	Drop columns

14. Rename new_grad_candidates to n_grad_candidates.

```
ALTER TABLE new_grad_candidates
```

```
RENAME TO n_grad_candidates;
```

✓ 25 12:18:31 ALTER TABLE new_grad_candidates RENAME TO n_grad_candidates 0 row(s) affected 0.031 sec

EXPERIMENT 4

AIM: - Execution of DML Commands

DML (Data Manipulation Language): A data manipulation language (DML) includes commands permitting users to manipulate data in a database. This manipulation involves inserting data into database tables, retrieving existing data, deleting data from existing tables and modifying existing data. DML is mostly incorporated in SQL database

List of DML Commands in SQL:

Here is a shortlist of all DML commands and their specific functions in the SQL programming language.

SELECT: Command to fetch data or values from the database

INSERT: Command to add new or fresh value to the database

UPDATE: Command to change or update the present/existing data to a newer value inside the database

DELETE: Command to remove or delete the values or data information from the database's current table

1) **SELECT:** Select is the most commonly used statement in SQL. The SELECT Statement in SQL is used to retrieve or fetch data from a database. We can fetch either the entire table or according to some specified rules. The data returned is stored in a result table. This result table is also called result-set.

With the SELECT clause of a SELECT command statement, we specify the columns that we want to be displayed in the query result and, optionally, which column headings we prefer to see above the result table.

Capabilities of the SELECT Statement

Data retrieval from data base is done through appropriate and efficient use of SQL. Three concepts from relational theory encompass the capability of the SELECT statement: projection, selection, and joining.

Projection: A project operation selects only certain columns (fields) from a table. The result table has a subset of the available columns and can include anything from a single column to all available columns.

Selection: A select operation selects a subset of rows (records) in a table (relation) that satisfy a selection condition. The ability to select rows from out of complete result set is called Selection. It involves conditional filtering and data staging. The subset can range from no rows, if none of the rows satisfy the selection condition, to all rows in a table.

Joining: A join operation combines data from two or more tables based on one or more common column values. A join operation enables an information system user to process the relationships that exist between tables. The join operation is very powerful because it allows system users to investigate relationships among data elements that might not be anticipated at the time that a database is designed.

2) INSERT INTO: The INSERT INTO statement of SQL is used to insert a new row in a table. There are two ways of using INSERT INTO statement for inserting rows:

Only values: First method is to specify only the value of data to be inserted without the column names.

```
INSERT INTO table_name VALUES (value1, value2, value3,);
```

table_name: name of the table.

value1, value2,; value of first column, second column,... for the new record

Column names and values both: In the second method we will specify both the columns which we want to fill and their corresponding values as shown below:

```
INSERT INTO table_name (column1, column2, column3,..) VALUES ( value1, value2, value3,..);
```

table_name: name of the table.

column1: name of first column, second column ...

value1, value2, value3 : value of first column, second column,... for the new record.

3) UPDATE Statement: The UPDATE statement in SQL is used to update the data of an existing table in database. We can update single columns as well as multiple columns using UPDATE statement as per our requirement.

Basic Syntax:

```
UPDATE table_name SET column1 = value1, column2 = value2,...
```

```
WHERE condition;
```

table_name: name of the table

column1: name of first, second, third column....

value1: new value for first, second, third column....

condition: condition to select the rows for which the values of columns needs to be updated.

NOTE: In the above query the SET statement is used to set new values to the particular column and the **WHERE** clause is used to select the rows for which the columns are needed to be updated. If we have not used the WHERE clause then the columns in all the rows will be updated. **So the WHERE clause is used to choose the particular rows.**

4) DELETE Statement: The DELETE Statement in SQL is used to delete existing records from a table. We can delete a single record or multiple records depending on the condition we specify in the WHERE clause.

Basic Syntax:

```
DELETE FROM table_name WHERE some_condition;
```

table_name: name of the table

some_condition: condition to choose particular record.

Note: We can delete single as well as multiple records depending on the condition we provide in WHERE clause. If we omit the WHERE clause then all of the records will be deleted and the table will be empty.

Write the SQL queries for the following:

1. Create a table named Student with the following fields:

- RollNo (Integer, Primary Key)
- Name (Varchar(50))
- Course (Varchar(50))
- Marks (Integer)

```
CREATE TABLE Student (  
    RollNo INTEGER PRIMARY KEY,  
    Name VARCHAR(50),  
    Course VARCHAR(50),  
    Marks INTEGER  
);
```

2. Insert five records into the student table with suitable data.

```
INSERT INTO Student (RollNo, Name, Course, Marks) VALUES (101, 'Anjali  
Sharma', 'Computer Science', 92);
```

```
INSERT INTO Student (RollNo, Name, Course, Marks) VALUES (102, 'Rahul  
Verma', 'Electrical Engineering', 85);
```

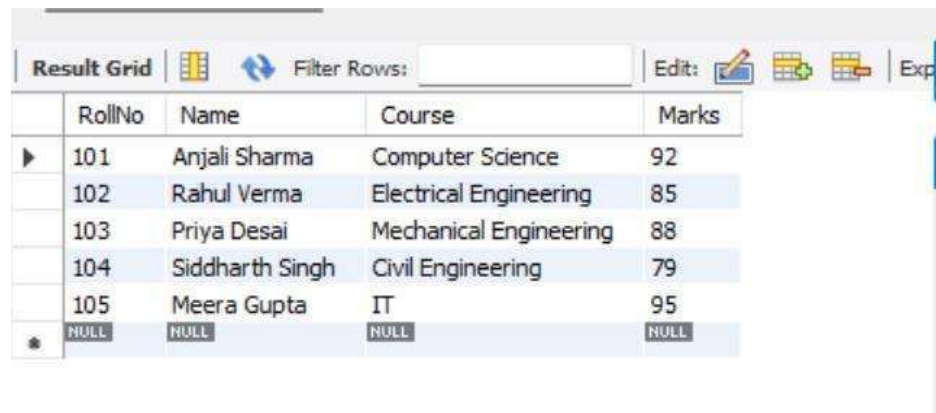
```
INSERT INTO Student (RollNo, Name, Course, Marks) VALUES (103, 'Priya  
Desai', 'Mechanical Engineering', 88);
```

```
INSERT INTO Student (RollNo, Name, Course, Marks) VALUES (104,  
'Siddharth Singh', 'Civil Engineering', 79);
```

```
INSERT INTO Student (RollNo, Name, Course, Marks) VALUES (105, 'Meera  
Gupta', 'IT', 95);
```

3. Display all the records of the Student table using the SELECT command.

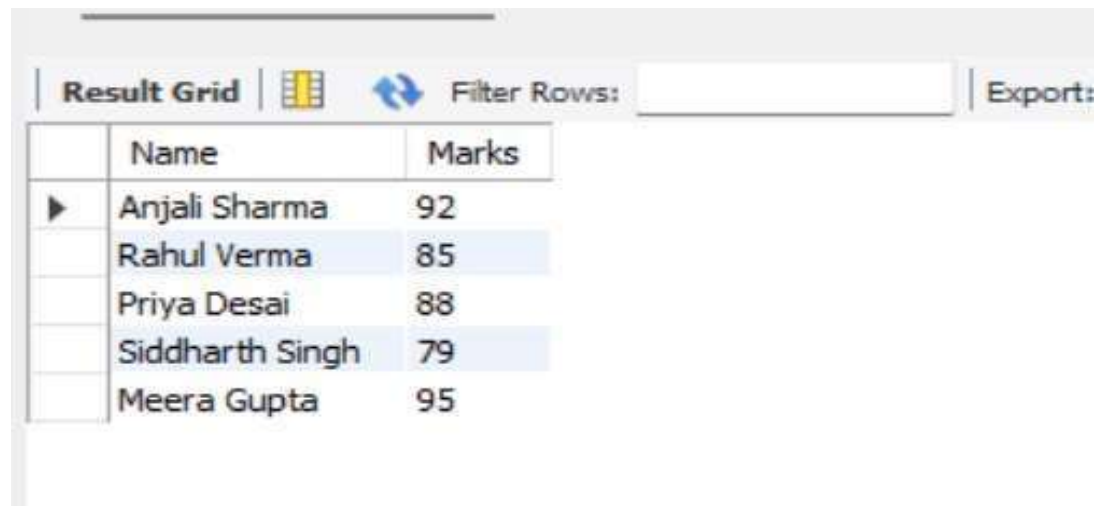
SELECT * FROM Student;



	RollNo	Name	Course	Marks
▶	101	Anjali Sharma	Computer Science	92
	102	Rahul Verma	Electrical Engineering	85
	103	Priya Desai	Mechanical Engineering	88
	104	Siddharth Singh	Civil Engineering	79
	105	Meera Gupta	IT	95
★	NULL	NULL	NULL	NULL

4. Display only the Name and Marks of all students.

SELECT Name, Marks FROM Student;



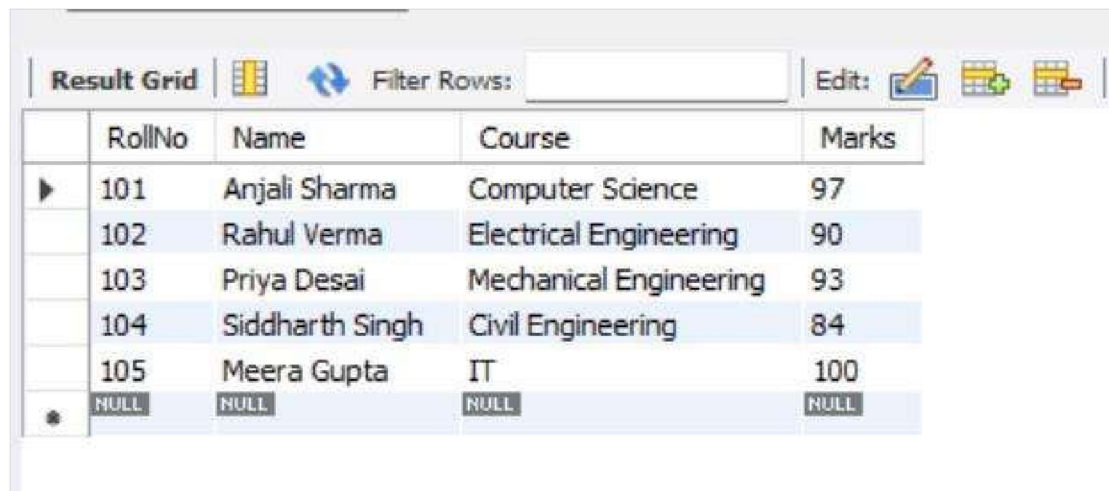
	Name	Marks
▶	Anjali Sharma	92
	Rahul Verma	85
	Priya Desai	88
	Siddharth Singh	79
	Meera Gupta	95

5. Update the marks of a student whose RollNo = 3.

```
UPDATE Student
SET Marks = 95
WHERE RollNo = 3;
```

6. Increase the marks of all students by 5 using a single SQL query.

```
UPDATE Student
SET Marks = Marks + 5;
```



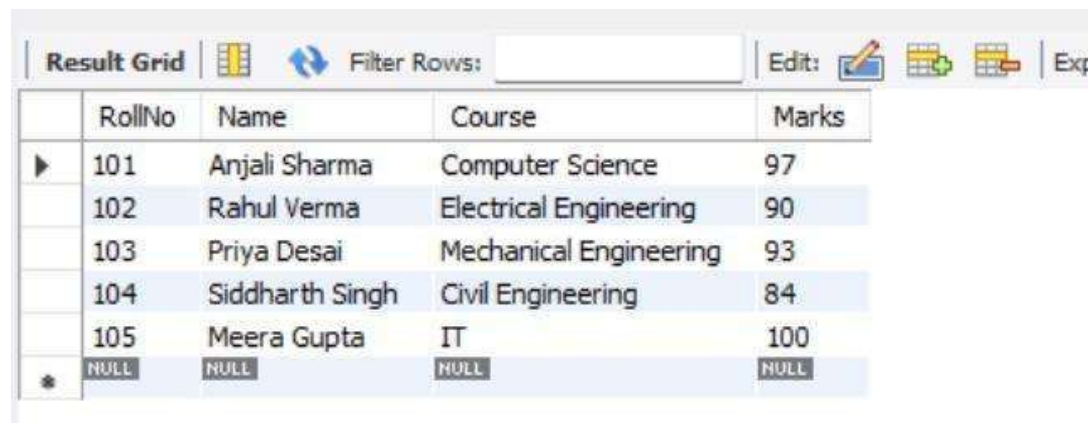
	RollNo	Name	Course	Marks
▶	101	Anjali Sharma	Computer Science	97
	102	Rahul Verma	Electrical Engineering	90
	103	Priya Desai	Mechanical Engineering	93
	104	Siddharth Singh	Civil Engineering	84
	105	Meera Gupta	IT	100
*	NULL	NULL	NULL	NULL

7. Delete the record of the student whose RollNo = 2.

```
DELETE FROM Student
WHERE RollNo = 2;
```

8. Delete all students whose marks are less than 50.

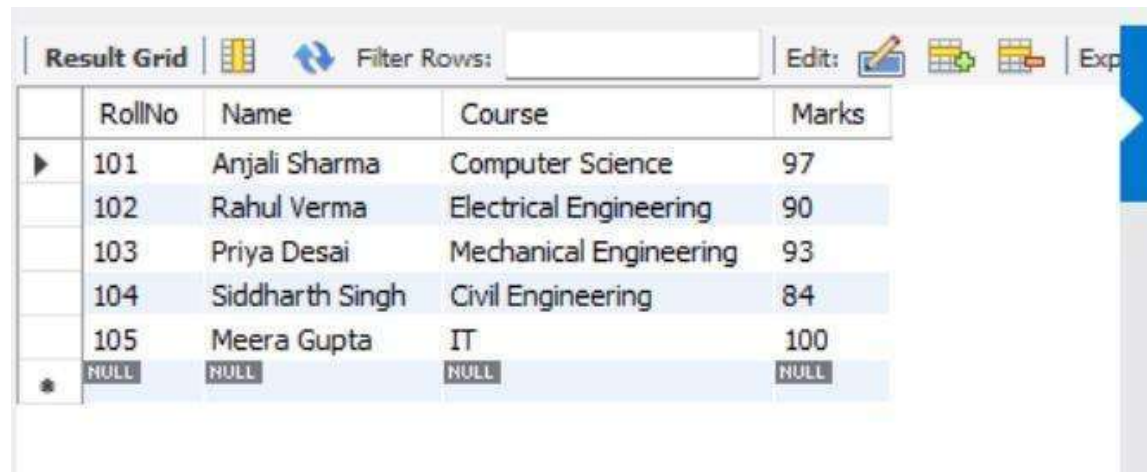
```
DELETE FROM Student
WHERE Marks < 50;
```



	RollNo	Name	Course	Marks
▶	101	Anjali Sharma	Computer Science	97
	102	Rahul Verma	Electrical Engineering	90
	103	Priya Desai	Mechanical Engineering	93
	104	Siddharth Singh	Civil Engineering	84
	105	Meera Gupta	IT	100
*	NULL	NULL	NULL	NULL

9. Display the records of students whose marks are greater than 80.

```
SELECT * FROM Student
WHERE Marks > 80;
```

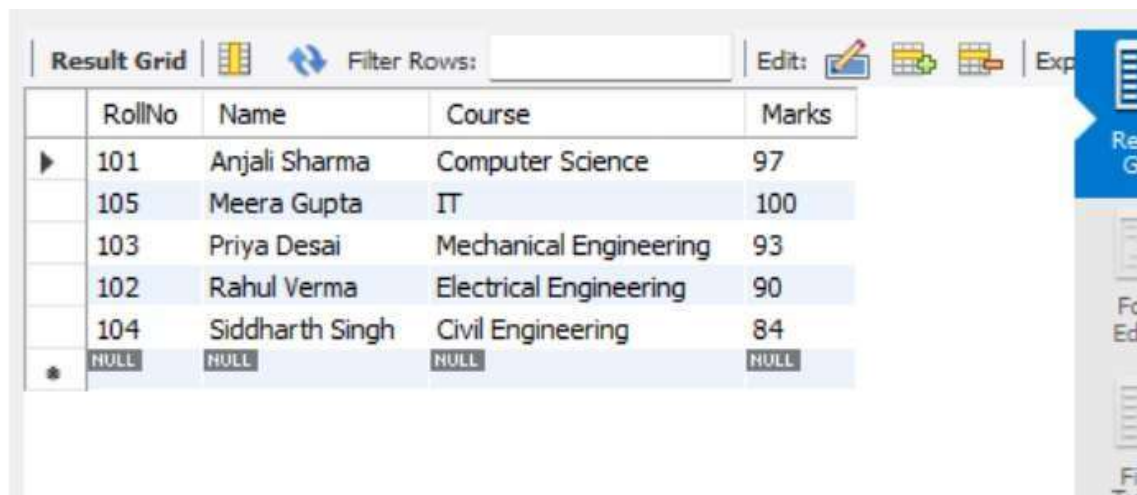


The screenshot shows a database application interface. At the top, there is a 'Result Grid' tab, a toolbar with icons for grid, refresh, filter, edit, and export, and a 'Filter Rows:' input field. Below this is a table with the following data:

	RollNo	Name	Course	Marks
▶	101	Anjali Sharma	Computer Science	97
	102	Rahul Verma	Electrical Engineering	90
	103	Priya Desai	Mechanical Engineering	93
	104	Siddharth Singh	Civil Engineering	84
	105	Meera Gupta	IT	100
*	NULL	NULL	NULL	NULL

10. Display all students in alphabetical order of their names using the ORDER BY clause.

```
SELECT * FROM Student
ORDER BY Name ASC;
```



The screenshot shows the same database application interface as before, but the table data is sorted alphabetically by name. The data is as follows:

	RollNo	Name	Course	Marks
▶	101	Anjali Sharma	Computer Science	97
	105	Meera Gupta	IT	100
	103	Priya Desai	Mechanical Engineering	93
	102	Rahul Verma	Electrical Engineering	90
	104	Siddharth Singh	Civil Engineering	84
*	NULL	NULL	NULL	NULL